

tf.compat.v1.Session Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/Session

A class for running TensorFlow operations.

Function Signatures

```
tf.compat.v1.Session( target='', graph=None, config=None )
```

Code Examples

```
tf.compat.v1.Session(  
target='', graph=None, config=None  
)
```

```
tf.compat.v1.Session(  
target='', graph=None, config=None  
)
```

```
tf.compat.v1.disable_eager_execution() # need to disable eager in
TF2.x
# Build a graph.
a = tf.constant(5.0)
b = tf.constant(6.0)
c = a * b
# Launch the graph in a session.
sess = tf.compat.v1.Session()
# Evaluate the tensor `c`.
print(sess.run(c)) # prints 30.0
```

```
tf.compat.v1.disable_eager_execution() # need to disable eager in
TF2.x
# Build a graph.
a = tf.constant(5.0)
b = tf.constant(6.0)
c = a * b
# Launch the graph in a session.
sess = tf.compat.v1.Session()
# Evaluate the tensor `c`.
print(sess.run(c)) # prints 30.0
```

```
# Using the `close()` method.
sess = tf.compat.v1.Session()
sess.run(...)
sess.close()
# Using the context manager.
with tf.compat.v1.Session() as sess:
    sess.run(...)
```

```
# Using the `close()` method.
sess = tf.compat.v1.Session()
sess.run(...)
sess.close()
# Using the context manager.
with tf.compat.v1.Session() as sess:
    sess.run(...)
```

```
# Launch the graph in a session that allows soft device placement
and
# logs the placement decisions.
sess = tf.compat.v1.Session(config=tf.compat.v1.ConfigProto(
    allow_soft_placement=True,
    log_device_placement=True))
```

```
# Launch the graph in a session that allows soft device placement
and
# logs the placement decisions.
sess = tf.compat.v1.Session(config=tf.compat.v1.ConfigProto(
    allow_soft_placement=True,
    log_device_placement=True))
```

Documentation

A class for running TensorFlow operations.

Migrate to TF2

Session does not work with either eager execution or `tf.function`, and you should not invoke it directly. To migrate code that uses sessions to TF2, rewrite the code without it. See the [migration guide](#) on replacing `Session.run` calls.

Description

Used in the notebooks

- Migrating model checkpoints
- Validating correctness & numerical equivalence

- Migrate the SavedModel workflow
- Debug a TensorFlow 2 migrated training pipeline
- Migrating your TFLite code to TF2
- Universal Sentence Encoder-Lite demo
- Generating Images with Little Data Using S3GAN
- Generating Images with BigBiGAN
- Generating Images with BigGAN
- Exploring the TF-Hub CORD-19 Swivel Embeddings

A Session object encapsulates the environment in which Operation objects are executed, and Tensor objects are evaluated. For example:

A session may own resources, such as `tf.Variable`, `tf.queue.QueueBase`, and `tf.compat.v1.ReaderBase`. It is important to release these resources when they are no longer required. To do this, either invoke the `tf.Session.close` method on the session, or use the session as a context manager. The following two examples are equivalent:

The `ConfigProto` protocol buffer exposes various configuration options for a session. For example, to create a session that uses soft constraints for device placement, and log the resulting placement decisions, create a session as follows:

Attributes

Methods

as_default

[View source](#)

Returns a context manager that makes this object the default session.

Use with the with keyword to specify that calls to `tf.Operation.run` or `tf.Tensor.eval` should be executed in this session.

To get the current default session, use `tf.compat.v1.get_default_session`.

Alternatively, you can use with `tf.compat.v1.Session()`: to create a session that is automatically closed on exiting the context, including when an uncaught exception is raised.

[View source](#)

Closes this session.

Calling this method frees all resources associated with the session.

list_devices

[View source](#)

Lists available devices in this session.

make_callable

[View source](#)

Returns a Python callable that runs a particular step.

The returned callable will take `len(feed_list)` arguments whose types must be compatible feed values for the respective elements of `feed_list`. For example, if element `i` of `feed_list` is a `tf.Tensor`, the `i`th argument to the returned callable must be a numpy ndarray (or something convertible to an ndarray) with matching element type and shape. See `tf.Session.run` for details of the allowable feed key and value types.

The returned callable will have the same return type as `tf.Session.run(fetches, ...)`. For example, if `fetches` is a `tf.Tensor`, the callable will return a numpy ndarray; if `fetches` is a `tf.Operation`, it will return `None`.

partial_run

[View source](#)

Continues the execution with more feeds and fetches. (deprecated)

This is EXPERIMENTAL and subject to change.

To use partial execution, a user first calls `partial_run_setup()` and then a sequence of `partial_run()`. `partial_run_setup` specifies the list of feeds and fetches that will be used in the subsequent `partial_run` calls.

The optional `feed_dict` argument allows the caller to override the value of tensors in the graph. See `run()` for more information.

Below is a simple example:

partial_run_setup

[View source](#)

Sets up a graph with feeds and fetches for partial run. (deprecated)

This is EXPERIMENTAL and subject to change.

Note that contrary to `run`, `feeds` only specifies the graph elements. The tensors will be supplied by the subsequent `partial_run` calls.

[View source](#)

Resets resource containers on target, and close all connected sessions.

A resource container is distributed across all workers in the same cluster as target. When a resource container on target is reset, resources associated with that container will be cleared. In particular, all Variables in the container will become undefined: they lose their values and shapes.

- (i) `reset()` is currently only implemented for distributed sessions.
- (ii) Any sessions on the master named by target will be closed.

If no resource containers are provided, all containers are reset.

[View source](#)

Runs operations and evaluates tensors in fetches.

This method runs one "step" of TensorFlow computation, by running the necessary graph fragment to execute every Operation and evaluate every Tensor in fetches, substituting the values in feed_dict for the corresponding input values.

The fetches argument may be a single graph element, or an arbitrarily nested list, tuple, namedtuple, dict, or OrderedDict containing graph elements at its leaves. A graph element can be one of the following types:

- A tf.Operation.

The corresponding fetched value will be None.

- A tf.Tensor.

The corresponding fetched value will be a numpy ndarray containing the value of that tensor.

- A tf.sparse.SparseTensor.

The corresponding fetched value will be a tf.compat.v1.SparseTensorValue containing the value of that sparse tensor.

- A get_tensor_handle op. The corresponding fetched value will be a

numpy ndarray containing the handle of that tensor.

- A string which is the name of a tensor or operation in the graph.

The value returned by `run()` has the same shape as the `fetches` argument, where the leaves are replaced by the corresponding values returned by TensorFlow.

Example:

The optional `feed_dict` argument allows the caller to override the value of tensors in the graph. Each key in `feed_dict` can be one of the following types:

- If the key is a `tf.Tensor`, the

value may be a Python scalar, string, list, or numpy ndarray that can be converted to the same dtype as that tensor. Additionally, if the key is a `tf.compat.v1.placeholder`, the shape of the value will be checked for compatibility with the placeholder.

- If the key is a

`tf.sparse.SparseTensor`,
the value should be a
`tf.compat.v1.SparseTensorValue`.

- If the key is a nested tuple of `Tenso`